

Systematic Literature Review Report

1. Igor Rogoziński, s193326
2. Jan Szczepkowski, s193667
3. Damian Trowski, s193443
4. Mateusz Fydrych, s193410

1. Research project

- **1.1. Title:** The influence of LLM agents on the development of open-source projects. (Wpływ agentów LLM na rozwój projektów open-source)
- **1.2. Supervisor:** mgr inż. Jan Majkutewicz (KASK)
- **1.3. Goals and short description:**

The project's goal is to examine how LLM-based tools such as Codex and Claude Code, as well as autonomous agents such as OpenClaw, impact the development of open-source projects.

The project will involve monitoring selected open-source projects and comparing their current metrics with historical data.

2. Systematic Literature Review plan

- **2.1. Goals and questions:** (*Goals of SLR and initial research questions to try to answer with SLR*)
 - **GOAL1:** *To identify and validate the performance metrics (e.g., time-to-merge, commit frequency) used in existing literature, providing a scientific foundation for the project's measurement methodology.*
 - **RQ1:** *What specific velocity metrics are most commonly used in academic literature to evaluate the impact of LLMs on Open Source projects?*
 - **RQ2:** *What is the reported impact of AI assistants (e.g., Copilot, Cursor) on the lifecycle of issues and PRs in the Open Source ecosystem according to current studies?*
- **2.2. Keywords:** (*Keywords to search for, with synonyms*)
 - **Open-source**
 - OSS (Open Source Software)
 - Open Source Software
 - GitHub
 - Public Repositories
 - FOSS (Free and Open Source Software)
 - **LLM**

- Large Language Models
 - Generative AI
 - GitHub Copilot
 - Cursor
 - AI Coding Assistant
 - AI Agents
 - Codex
 - Claude Code
- **Velocity**
 - Productivity
 - Development Speed
 - Time-to-merge
 - Lead Time
 - Cycle Time
 - Issue resolution
- **2.3. Search strings:** *(Fragment and final search strings, possible customizations for each database)*
 - **Fragmentary search strings:**
 - ("Open Source" OR "OSS" OR "FOSS" OR "GitHub" OR "Public Repository")
 - ("LLM" OR "Large Language Model" OR "Generative AI" OR "GitHub Copilot" OR "AI Agent" OR "Codex" OR "Claude Code")
 - ("Velocity" OR "Productivity" OR "Time-to-merge" OR "Lead Time" OR "Cycle Time" OR "Contribution" OR "Contributor")
 - **Final search strings:**
 - ("Open Source" OR "OSS" OR "FOSS" OR "GitHub") AND ("LLM" OR "Large Language Model" OR "Generative AI" OR "GitHub Copilot" OR "AI Agent" OR "Claude Code") AND ("Velocity" OR "Productivity" OR "Time-to-merge" OR "Contributor")
- **2.4. Literature databases:** *(Scientific literature databases selected for SLR search)*
 - Scopus
 - IEEE Xplore
 - ACM
- **2.5. Inclusion criteria:** *(Publication year, type, language, scope etc. of articles to include in SLR results)*
 - **IC1 (Year):** Publications from 2020 to 2026. (Rationale: Significant research on LLMs in software engineering, such as OpenAI Codex or early Copilot studies, emerged after 2020).
 - **IC2 (Subject area):** Computer Science.
 - **IC3 (Document type):** Article, Conference paper, Review.
 - **IC4 (Language):** Articles written exclusively in English.
 - **IC5 (Publication stage):** Final.

- **2.6. Exclusion criteria:** *(Any criteria that eliminate the articles from SLR results e.g. lack of industrial validation, empirical results etc.)*
 - **EC1 (Out of Scope):** Research that focuses on AI in non-software domains (e.g., medical diagnostics, financial forecasting) or strictly closed-source enterprise environments that do not share open-source characteristics.
 - **EC2 (Lack of Empirical Evidence):** Position papers, editorials, opinion pieces, or purely theoretical articles that do not provide experimental data, case studies, or quantitative measurements.
 - **EC3 (Incomplete or Short Format):** Extended abstracts, posters, slide decks, or papers shorter than 3 pages that lack a detailed description of the research methodology.
 - **EC4 (Legacy Technologies):** Studies focusing on rule-based automation or "classic" code completion (e.g., IntelliSense) rather than modern Transformer-based LLMs or AI agents.
 - **EC5 (Redundancy):** Duplicate entries of the same study found across multiple databases (only the most complete version will be kept).
 - **EC6 (Redundancy):** Closed access.
- **2.7. Quality criteria:** *(Research method, sample size, research data etc. of articles to evaluate and qualify for data extraction)*
 - Strict quality criteria were not applied, as the objective was to capture a comprehensive overview of emerging research in a rapidly evolving field. Imposing rigid constraints on methodology or sample size could have led to the exclusion of relevant and insightful studies.
- **2.8. Data extraction:** *(What research data will be extracted from selected articles)*
 - **Title**
 - **Keywords**
 - **LLM agents analyzed**
 - **Repositories analyzed**
 - **Conclusions**
 - **Year of publication**
- **2.9. SLR process:** *(Process steps, assignments to team members - execution, verification, tools to use)*
 - **Main process steps and assignments:**
 - Planning: Conducted jointly by all team members.
 - Search execution: Performed using PRIMO, Scopus, and Zotero - Jan Szczepkowski.
 - Duplicate removal: Performed in google sheets and manually verified - Damian Trowski.

- Screening (title and abstract) - conducted together. Articles are split equally among team members.
- Validation of screening decisions: Each decision is reviewed by another team member. In case of disagreement, a third member makes the final decision.
- Full-text reading - Performed asynchronously by each team member.
- Final selection of articles - full reading. Conducted asynchronously by each team member. Articles are split equally among team members.
- Data extraction - performed asynchronously by the readers.
- Reporting - Conducted jointly by all team members.

○ **Tools to be used:**

- Zotero (reference management and deduplication)
- Google Sheets (data extraction and tracking)
- Google Docs (collaborative writing)
- PRIMO (search engine)
- Scopus (literature search database)

3. Systematic Literature Review results

● **3.1. Results in numbers:** (Number of articles in each SLR phase from each database)

○ **Raw data:** 244

	Count
Scopus	155
IEEE	61
ACM	22
Sum	244

○ **After removing duplicates (by DOI): 158 / After selection (screening): 38**

	Include	Exclude	Sum
Scopus	37	98	135
IEEE	0	15	15
ACM	1	7	8
Sum	38	120	158

○ **After reading articles:** 11

	Count
Scopus	10
IEEE	0

ACM	1
Sum	11

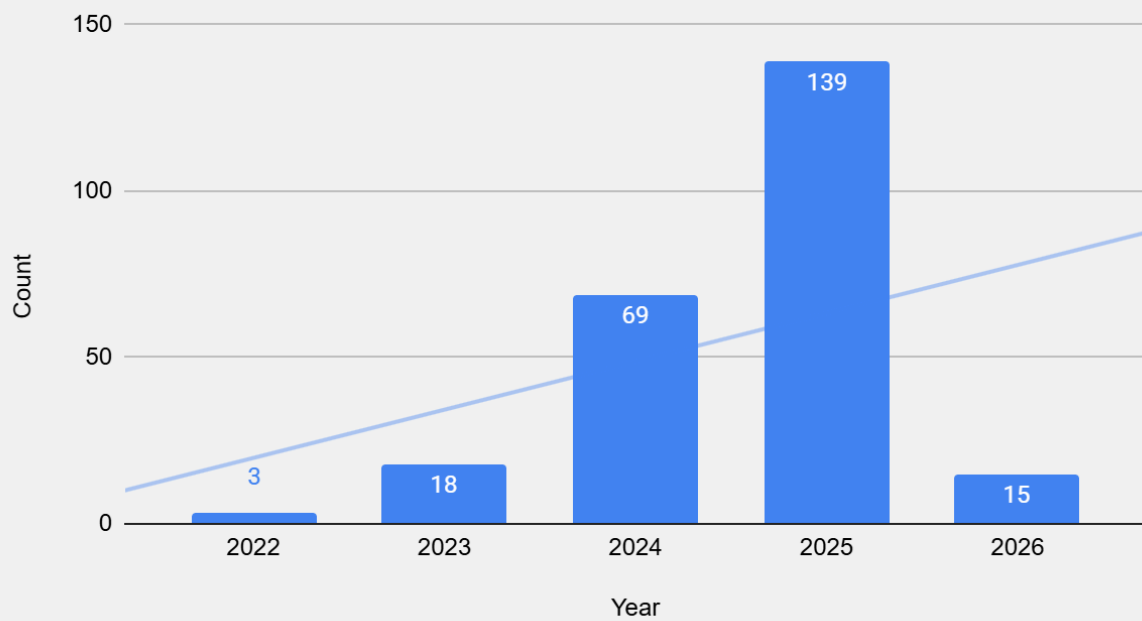
- **3.2. Articles selected for data extraction:** (List of articles from search results qualifying for data extraction according to inclusion, exclusion and quality criteria)

Based on inclusion, exclusion and quality criteria, 11 publications were selected for further review.

1. Productivity Assessment of Neural Code Completion
2. A Static Evaluation of Code Completion by Large Language Models
3. A Journey Through SPACE: Unpacking the Perceived Productivity of GitHub Copilot
4. Reading Between the Lines: Modeling User Behavior and Costs in AI-Assisted Programming
5. Assessing output reliability and similarity of large language models in software development: A comparative case study approach
6. Establishing Traceability Between Release Notes and Software Artifacts: Practitioners' Perspectives and Automated Recovery using LLMs
7. Impact of code context and prompting strategies on automated unit test generation with modern general-purpose large language models
8. Enhancing Developer Productivity: Benchmarking LLM-Powered Tools like GitHub Copilot and TabNine in Real-Time Coding Environments
9. Students' Perspectives on AI Code Completion: Benefits and Challenges
10. The Impact of AI Tools on Software Development: A Case Study with GitHub Copilot and Other AI Assistants
11. The Impact of Large Language Models on Open-Source Innovation: Evidence from GitHub Copilot

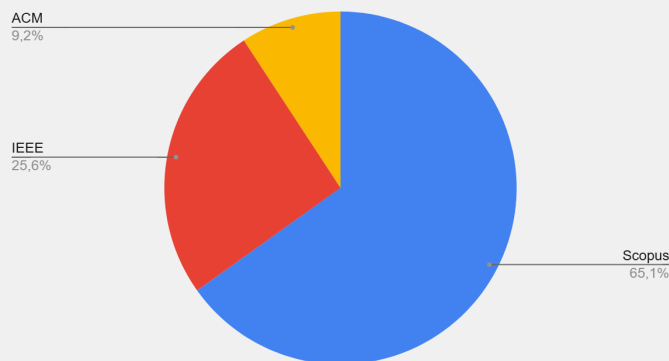
- **3.3. Snowballed articles:** (Additional articles found with snowballing technique)
 - Due to the sufficient number of articles identified, we decided not to conduct snowballing.
- **3.4. Article statistics:** (Charts, histograms of publication year, journal title, topic etc.)
 - **Number of publications by year:**

Publication count by year



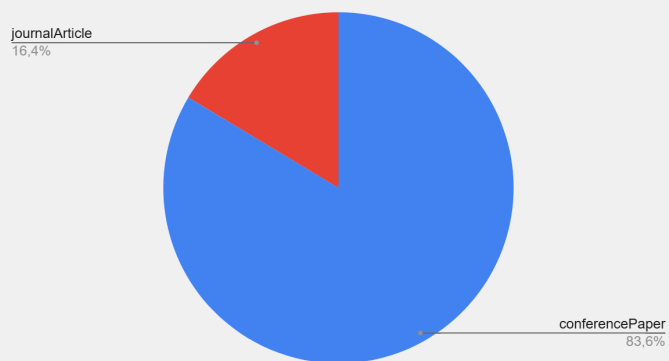
○ **Number of publications by data source:**

Count by source



○ **Number of publications by document type:**

Item Type



- **3.5. Initial extracted data:** (Initial set of research data extracted from SLR literature)

The table below presents data extracted from the selected articles. For each article, the main findings row spans the full width of the table.

Nr	Article Title	Keywords	LLM agents analyzed	Repositories analyzed	Year
1	Productivity Assessment of Neural Code Completion	code synthesis, code completion, neural networks, productivity, LLM evaluation	GitHub Copilot	Not specified	2022
	Acceptance rate (number of suggestions accepted divided by number shown) is the strongest predictor of perceived developer productivity. More detailed metrics such as persistence or number of characters contributed are less predictive. Neural code completion tools function as an interactive aid, providing value beyond simple correctness of suggestions. Findings highlight that developer-perceived productivity is best measured via user interaction metrics rather than traditional offline benchmarks.				
2	A Static Evaluation of Code Completion by Large Language Models	Categorization, Commit's Quality, Experiment, LLMs, Maintenance	ChatGPT-4	34 different repositories	2023
	LLMs (specifically GPT-4) can achieve significant accuracy in categorizing developer intent when guided by a structured taxonomy. However, the varying quality of human-written commit messages (13.99% labeled as "low quality") remains a major barrier to effective automation in open-source project management.				
3	A Journey Through SPACE: Unpacking the Perceived Productivity of GitHub Copilot	developer productivity, SPACE framework, AI Coding Assistants, and GitHub Copilot	GitHub Copilot	Not specified	2025
	The study concludes that while GitHub Copilot significantly improves perceived productivity and task completion speed, its impact on developer "flow" and job fulfillment is more conservative than previously reported. Ultimately, the research highlights that AI assistants excel at reducing tedious tasks, yet their overall benefit depends heavily on the specific dimensions of the SPACE framework being measured.				
4	Reading Between the Lines: Modeling User Behavior and Costs in AI-Assisted Programming	AI-assisted programming, Copilot, user behavior, programmer productivity, CUPS taxonomy, telemetry analysis	GitHub Copilot	Not specified	2024

	The study introduces CUPS (CodeRec User Programming States), a taxonomy of 12 programmer activities while interacting with AI code recommendation tools. Key insights include: programmers spend significant time verifying, editing, and crafting prompts for AI suggestions; traditional metrics like acceptance rate or task completion time underestimate AI interaction costs; telemetry-based segmentation allows precise profiling of programmer behavior; findings suggest new interface designs and productivity metrics should account for mental effort, prompt engineering, and verification tasks, not just code output.				
5	Assessing output reliability and similarity of large language models in software development: A comparative case study approach	LLMs, Software Development, Output reliability, Output similarity, Comparative analysis, Case study	ChatGPT, Claude, Copilot, Gemini, Meta	Food Order and Delivery System, Smart Wallet System. Names weren't specified	2025
	The study concludes that while Large Language Models demonstrate an overall reliability of 0.82, a significant 43% variability in their outputs highlights a lack of consistency across different platforms. Consequently, the authors emphasize that LLM-generated software artifacts still require rigorous human oversight and cannot yet be fully trusted without manual verification.				
6	Establishing Traceability Between Release Notes and Software Artifacts: Practitioners' Perspectives and Automated Recovery using LLMs	Software Traceability, Release Notes, Large Language Models (LLMs), Open-Source Software, Empirical Study, GitHub	Meta LLaMA 3.1 (8B-Instruct), Gemini 1.5 Pro	10 high-maturity open-source GitHub repositories	2025
	The study demonstrates that LLM-based approaches, particularly Gemini 1.5 Pro, significantly outperform traditional baselines by effectively bridging informational gaps between high-level release notes and technical development artifacts. By integrating textual reasoning with non-textual features like time proximity, these models provide a scalable solution to reduce documentation debt and satisfy the strong practitioner demand for automated traceability tools.				

7	Impact of code context and prompting strategies on automated unit test generation with modern general-purpose large language models	Automated unit test generation, Large language model, Software development, Software testing	GPT-4.5 (M1), GPT-o3 (M2), GPT-o4-mini-high (M3), Claude 3.7 Sonnet (M4), Gemini 2.5 Pro (M5), and DeepSeek-V3 (M6)	Not specified	2026
	The study concludes that providing full implementation context is essential for reducing AI hallucinations and achieving high branch coverage, with Gemini 2.5 Pro emerging as the top performer in fault detection. Furthermore, while sequential multi-turn prompting significantly improves test adequacy, it introduces a trade-off by being nearly three times slower than simple prompting and increasing test redundancy.				
8	Enhancing Developer Productivity: Benchmarking LLM-Powered Tools like GitHub Copilot and TabNine in Real-Time Coding Environments	Large Language Models (LLMs), Software Development, GitHub Copilot, TabNine, Developer Productivity, Benchmarking, Real-Time Coding	GitHub Copilot (OpenAI/Microsoft), TabNine	Not specified	2025
	The research concludes that GitHub Copilot generally offers superior productivity gains and feature richness compared to TabNine, particularly in automating boilerplate code and assisting with complex reasoning. However, the study emphasizes that while these tools significantly reduce development time, long-term workflow integration requires careful monitoring of code quality and developer reliance to ensure sustainable development practices.				
9	Students' Perspectives on AI Code Completion: Benefits and Challenges	AI Code Completion, StarCoder, AutoAurora, Visual Studio Code, Programming Education, LLM, Students' Perspectives, Productivity, Coding Tutor, Code Quality	StarCoder (used in AutoAurora extension)	Not specified	2024

	AI code completion enhances students' productivity and efficiency through syntax suggestions, alternative solutions, and guidance akin to a coding tutor. Key challenges include over-reliance, which may limit conceptual understanding and creativity, and complicate assessment. Students expect future tools to explain suggested code, assist in refactoring, adapt to personal coding style, support natural language input, and provide complete solution suggestions.				
10	The Impact of AI Tools on Software Development: A Case Study with GitHub Copilot and Other AI Assistants	ChatGPT, GitHub Copilot, Code Generation, Empirical Study, Software Engineering, Software Development, Artificial Intelligence	GitHub Copilot, ChatGPT, Gemini	Not specified	2025
	AI tools positively impact productivity, code quality, learning, and job satisfaction; enable focus on complex tasks by automating routine work; organizational support (licenses, training) is important; need for review and cautious use to avoid dependency; further comparative research on different AI assistants suggested				
11	The Impact of Large Language Models on Open-Source Innovation: Evidence from GitHub Copilot	Generative AI, Open-Source Community, Innovation, Productivity, Large Language Models	GitHub Copilot, GPT-4o	Python, R, Rust, Haskell	2024
	GitHub Copilot boosts open-source contributions, mainly by supporting iterative innovation (refining existing features) rather than capability innovation (creating new features). The effect is stronger in active, context-rich projects and grows as LLMs improve, shifting innovation toward exploitation over exploration.				

4. Conclusions

- **4.1. SLR process:** (Conclusions and lessons learnt about the SLR process executed)

The systematic literature review process provided a structured framework for navigating the rapidly expanding intersection of LLMs and open-source software. Several key lessons were learned during the execution:

- **The "Moving Target" Challenge:** Given that LLM technology evolves monthly, we found that traditional databases (Scopus/IEEE) sometimes lag behind the latest tool releases (e.g., Claude 3.7 or O3). A lesson learned is that for cutting-edge AI topics, the inclusion of **preprint servers (like arXiv)** or specific **industry-vetted white papers** might be necessary

to capture the most current state of the art, though it requires stricter manual quality control.

- **Refinement of Search Strings:** Initially, our search strings were too broad, leading to a high number of results (244) that were out of scope (EC1). We learned that incorporating specific tool names (e.g., "Copilot," "Claude Code") alongside general terms is vital to filter out general AI research that lacks application to Software Engineering.
- **Inter-rater Reliability:** The screening process highlighted the importance of the **"Validation of screening decisions"** step. Differences in how team members interpreted "Empirical Evidence" (EC2) were resolved through the third-party tie-breaker rule, which ensured the final selection of 39 articles maintained a high standard of academic rigor.
- **Database Overlap:** We observed a significant overlap (nearly 35%) between Scopus and IEEE. Using **Zotero** for automated deduplication was a critical time-saver, though manual verification remained necessary to catch entries with slightly different title formatting or missing DOIs.
- **Defining "Quality" in AI Research:** We learned that QA5 (distinguishing human vs. AI contributions) is the hardest criterion to meet. Many current studies rely on self-reporting or simulated environments because identifying AI-generated code in existing OSS history is technically difficult without metadata.

- **4.2. SLR results:** (Conclusions and lessons learnt about the results achieved with SLR)

The synthesis of the 11 analyzed articles provides a comprehensive overview of how LLM agents influence the open-source software (OSS) development lifecycle. The results are categorized into thematic findings that directly address the project's research questions.

- **Dominance of Velocity Metrics in Academic Literature (RQ1)**

The systematic review confirms that "Time-to-merge" and "Lead Time" are the most prevalent metrics used to evaluate LLM impact on developer velocity. Studies indicate that while simple boilerplate automation offers immediate speed gains (up to 50% reduction in coding time for routine tasks), these metrics must be balanced against "Test Adequacy". Furthermore, the literature highlights that velocity is highly sensitive to the "Code Context" provided; models integrated with full implementation details significantly outperform those relying on method signatures alone, reducing the "hallucination rate" that often leads to time-consuming debugging loops.

- **Transformation of the PR and Issue Lifecycle (RQ2)**

AI assistants like GitHub Copilot and specialized multi-agent frameworks (e.g., MAF-CPR) are fundamentally altering how PRs are handled. The research identifies three key shifts:

- **Automated Traceability:** LLMs have proven highly effective at recovering links between high-level documentation (release notes) and technical artifacts (commits and PRs), satisfying a strong practitioner demand for automated documentation tools.
- **Intelligent Code Review:** New "Agentic-AI" solutions are moving beyond static analysis to provide context-aware feedback during the review phase, though human-in-the-loop validation remains a requirement for adhering to quality standards like ISO/IEC 29110.

- **Issue Resolution Efficiency:** AI-powered duplicate bug report detection and task recommendation systems are reducing the manual triage burden in high-maturity repositories.

- **The Trust Calibration Paradox**

A critical finding regarding the human element of OSS development is the "Trust Calibration" process. Initial use of LLM agents often results in either over-reliance (leading to security vulnerabilities) or under-reliance (missing productivity opportunities). The literature suggests that after approximately 10 days of extended use, programmers typically reach an "optimal trust" level, shifting toward a "trust-but-verify" methodology that is essential for maintaining the integrity of open-source projects.

- **Performance Divergence Among LLM Providers**

The selection of the underlying model is not neutral to the development outcome. Benchmarking reveals that while GitHub Copilot (OpenAI/Microsoft) remains the leader in real-time IDE integration and feature richness, models like Gemini 1.5 Pro excel in tasks requiring reasoning across large context windows, such as complex traceability recovery and cross-file unit test generation.

- **Implications for the Research Project**

These findings validate the project's goal of monitoring **time-to-merge** and **commit frequency** as primary indicators of impact. The SLR results emphasize that our project should not only focus on "speed" but also on the "quality of integration"—specifically how autonomous agents like OpenClaw manage the transition from human-written to AI-assisted contributions, which remains the most difficult metric to verify (QA5).

5. Literature

1. Productivity Assessment of Neural Code Completion
2. A Static Evaluation of Code Completion by Large Language Models
3. A Journey Through SPACE: Unpacking the Perceived Productivity of GitHub Copilot
4. Reading Between the Lines: Modeling User Behavior and Costs in AI-Assisted Programming
5. Assessing output reliability and similarity of large language models in software development: A comparative case study approach
6. Establishing Traceability Between Release Notes and Software Artifacts: Practitioners' Perspectives and Automated Recovery using LLMs
7. Impact of code context and prompting strategies on automated unit test generation with modern general-purpose large language models
8. Enhancing Developer Productivity: Benchmarking LLM-Powered Tools like GitHub Copilot and TabNine in Real-Time Coding Environments
9. Students' Perspectives on AI Code Completion: Benefits and Challenges
10. The Impact of AI Tools on Software Development: A Case Study with GitHub Copilot and Other AI Assistants

11. The Impact of Large Language Models on Open-Source Innovation: Evidence from GitHub Copilot

MBN